

Interview Questions & Answers

Flights Booking App · PixelPing Extension · PixelPing Backend

Akshat Rauthan — Project Deep Dive

Interview Questions & Answers — Project Deep Dive

*Covers **Flights Booking App** (microservices) and **PixelPing** (email-tracking browser extension + backend). Questions are grouped by topic and difficulty. Each answer is comprehensive enough to use as a spoken answer in a technical interview.*

TABLE OF CONTENTS

1. [System Design & Microservices Architecture](#)
2. [API Gateway — Auth, RBAC & Rate Limiting](#)
3. [Message Queues — RabbitMQ & Async Communication](#)
4. [Caching — Redis Strategy](#)
5. [Resilience — Circuit Breakers & Retries](#)
6. [Booking Service — Transactions, Idempotency & Cron](#)
7. [Database Design — MySQL & Sequelize ORM](#)
8. [Input Validation & Security](#)
9. [Testing Strategy](#)
10. [CI/CD & Docker](#)
11. [Observability — Logging & Correlation IDs](#)

12. [PixelPing — Chrome Extension \(MV3\)](#)
 13. [PixelPing — Backend \(TypeScript + Express + MongoDB\)](#)
 14. [General Backend & Node.js Concepts](#)
 15. [HR / Behavioural Round](#)
-

1. System Design & Microservices Architecture

Q1. Walk me through the overall architecture of your Flights Booking App.

Answer: The application follows a **microservices architecture** with five independently deployable services:

Service	Port	Role
API Gateway	3001	Single entry point — JWT auth, RBAC, per-user Redis rate limiting, Zod validation, reverse proxy
Flights Booking Service	3002	Booking lifecycle — create, pay, cancel, expiry cron, RabbitMQ publisher
Flights Creation Service	3003	Admin CRUD for flights, airports, cities, airlines, airplanes
Flights Searching Service	3004	Redis-cached BFF proxy for flight search
Notifications Service	3005	RabbitMQ consumer that sends transactional emails via Nodemailer

All external traffic enters through the API Gateway on port 3001. The gateway validates the JWT, checks RBAC roles, enforces per-user rate limits using Redis, and then reverse-proxies the request to the correct downstream service. Services communicate inter-service either via HTTP (guarded by AES-256-GCM service tokens) or asynchronously through RabbitMQ.

Q2. Why did you choose microservices over a monolith for this project?

Answer: The primary learning goals were **scalability patterns** and **resilience engineering**. Microservices allowed me to:

1. **Scale independently** — The Searching Service (read-heavy) can scale horizontally without touching the Booking Service.
2. **Fault isolation** — A failure in the Notifications Service does not take down bookings.
3. **Technology diversity** — I can use Redis as a caching layer specifically for the Searching Service without coupling it to the entire app.
4. **CI/CD granularity** — Each service has its own Dockerfile and can be deployed independently.

The trade-off is **operational complexity** (distributed tracing, inter-service auth, eventual consistency). I handled these with correlation IDs, circuit breakers, and RabbitMQ for async decoupling.

Q3. How do services communicate with each other? Give a concrete example.

Answer: There are two communication patterns:

Synchronous (HTTP): When the Booking Service needs to validate a flight before creating a booking, it calls `POST /api/v1/flights/validate` on the Creation Service. The request carries an AES-256-GCM encrypted service token so the Creation Service can verify the caller is internal. The Booking Service wraps this call inside an **Opossum circuit breaker**, so if the Creation Service is slow or down, the circuit opens and a fallback is returned instead of letting the thread hang.

Asynchronous (RabbitMQ): After a successful payment, the Booking Service publishes a `booking_confirmed` event to RabbitMQ. The Notifications Service consumes this event and sends an email. This is fire-and-forget — the Booking Service does not wait for the email to be sent. If the Notifications Service is down, RabbitMQ holds the message until it recovers (with a dead-letter queue for retries).

Q4. What are the main challenges in microservices that you encountered?

Answer: 1. **Distributed transactions:** When a booking is created, seats must be reserved on the Creation Service and a record inserted in the Booking Service's DB. I used a **saga-like compensating approach** — if payment fails, a cancel event is published to restore seats. 2. **Service discovery:** In the current implementation services are addressed by static hostnames

(Docker Compose service names), which works locally but would need a service registry (Consul/Kubernetes DNS) in production. 3. **Debugging across services:** Without correlation IDs, tracing a request through five services is impossible. I generate a UUID at the gateway and propagate it in the `x-correlation-id` header so all log lines for a single request are linkable. 4. **Data consistency:** Each service owns its own database (Database-per-Service pattern). The Booking DB does not JOIN with the Flights DB — it caches the data it needs or calls the other service.

Q5. What is the BFF (Backend-for-Frontend) pattern and how did you use it?

Answer: BFF stands for **Backend For Frontend**. Instead of the client calling the Flights Creation Service directly for every search query, I created a dedicated **Flights Searching Service** that: 1. Accepts the search request from the client. 2. Checks Redis for a cached response (cache key = serialized query params). 3. On cache miss, proxies the request to the Creation Service, stores the result in Redis with a TTL, and returns it.

This offloads read pressure from the Creation Service and drastically reduces response time for repeated queries. The Searching Service acts as an intermediary that is optimized specifically for the client's read use-case, which is the essence of BFF.

2. API Gateway — Auth, RBAC & Rate Limiting

Q6. How is JWT authentication implemented in the gateway?

Answer: On `POST /api/v1/auth/signin`, the user's password is verified with **bcrypt** and a signed JWT is returned:

```
jwt.sign({ id: user.id, role: user.role }, JWT_SECRET, { expiresIn: JWT_EXPIRY })
```

For protected routes, the `checkAuth` middleware extracts the token from the `Authorization: Bearer <token>` header, calls `jwt.verify()`, and attaches the decoded payload to `req.user`. If the token is missing or expired, a 401 is returned immediately before the request reaches the downstream service.

Q7. Explain RBAC — what roles exist and how is it enforced?

Answer: There are three roles: `system_admin`, `airline_admin`, and `customer`.

Authorization is enforced in the gateway with a `checkRole(requiredRole)` middleware that checks `req.user.role` against the required role. For example, creating a flight requires `airline_admin`, while booking a seat requires `customer`.

RBAC is enforced at the **gateway level**, not inside each microservice. The downstream services trust that the gateway has already validated the JWT and role — they only verify the **service token** to confirm the request originated from the gateway, not a direct call.

Q8. How is per-user rate limiting implemented with Redis?

Answer: I use a sliding window counter stored in Redis. The rate limiter middleware: 1. Extracts the user ID from `req.user.id` (falls back to `req.ip` for unauthenticated routes). 2. Builds a Redis key: `rate_limit:${userId}`. 3. Atomically increments the counter and sets an expiry using `INCR + EXPIRE` (or a single `SET ... EX ... NX`). 4. If the counter exceeds the limit (e.g., 100 requests per 15 minutes), responds with **429 Too Many Requests**.

The per-user keying is important — in a monolithic rate limiter keyed by IP, multiple users sharing a NAT/proxy would share one quota. Per-user keying is fairer and harder to bypass.

Q9. What is AES-256-GCM and why did you use it for service-to-service auth?

Answer: AES-256-GCM is a **symmetric authenticated encryption** algorithm.

"GCM" (Galois/Counter Mode) provides both confidentiality (the payload is encrypted) and integrity (any tampering with the ciphertext is detected via the auth tag).

For inter-service tokens I encrypt a JSON payload `{ service: 'booking', issuedAt, exp }` with a shared `SERVICE_ENCRYPTION_KEY` that is only known to the gateway and the creation service. When the Creation Service receives a request it decrypts the token; if decryption fails or the token is expired it rejects the request with 403. This prevents a rogue process from calling internal endpoints directly.

Using JWT for service tokens would also work, but JWT uses asymmetric or HMAC signing (not encryption), so the payload is base64-readable. AES-GCM keeps the payload opaque.

3. Message Queues — RabbitMQ & Async Communication

Q10. Why use RabbitMQ instead of calling the Notifications Service directly over HTTP?

Answer: HTTP is **synchronous and tightly coupled**. If the Notifications Service is down during a booking, the booking would fail too — a bad user experience.

With RabbitMQ: - **Decoupling:** The Booking Service publishes a message and moves on. It doesn't care whether the email is sent immediately. - **Durability:** The message persists in the queue even if the Notifications Service crashes and restarts. - **Retry semantics:** Failed deliveries can be moved to a **dead-letter queue (DLQ)** and retried. - **Backpressure:** If the Notifications Service is busy, messages queue up without blocking the Booking Service.

The trade-off is **eventual consistency** — the user may not receive the email immediately after booking. For a non-critical notification this is acceptable.

Q11. What is a Dead Letter Queue and how did you implement retry logic?

Answer: A Dead Letter Queue (DLQ) is a queue where messages are routed when they cannot be processed — for example, after `n` failed delivery attempts or when a message TTL expires.

In the Notifications Service, if `nodemailer.sendMail()` throws (e.g., SMTP timeout), the service catches the error, stores a failed ticket record in the database with `status: 'FAILED'`, and does **not** re-queue via RabbitMQ for that attempt. A separate **node-cron** job runs every 10 minutes, queries the DB for `status = 'FAILED'` tickets, and attempts to resend them. If it succeeds, the ticket status is updated to `SENT`.

This is a simpler alternative to native DLQ configuration — it gives full control and visibility into failed notifications without needing `x-dead-letter-exchange` RabbitMQ setup.

Q12. Explain exactly what happens in the system when a user makes a booking.

Answer: End-to-end booking flow:

1. **Client** → **API Gateway (POST /flightsBookingsService/api/v1/bookings)**
2. Gateway validates JWT, checks `customer` role, enforces rate limit.
3. Zod validates the request body (flightId, seats, userId).
4. Gateway proxies to Booking Service.
5. **Booking Service** — `createBooking()`

6. Calls `POST /api/v1/flights/validate` on Creation Service (via circuit breaker) to confirm the flight exists and has available seats.
 7. Calls `POST /api/v1/seats/bookings` on itself to atomically decrement seats (uses DB transaction).
 8. Inserts a Booking record with status `INITIATED`.
 9. Returns `bookingId` to the client.
 10. **Client → Booking Service (POST /api/v1/bookings/payments)**
 11. Client sends `bookingId` + `idempotencyKey`.
 12. The service checks if the `idempotencyKey` was already processed (to prevent duplicate charges).
 13. If not, updates booking status to `BOOKED` and publishes `booking_confirmed` event to RabbitMQ.
 14. **Notifications Service consumes the event**
 15. Sends confirmation email via Gmail SMTP.
 16. Updates ticket status to `SENT`.
-

4. Caching — Redis Strategy

Q13. How does the Redis cache in the Searching Service work?

Answer: The Searching Service implements a **cache-aside (lazy loading)** strategy:

1. Build a deterministic cache key from the query parameters (e.g., `flights:DEL:BOM:2024-12-25:economy`).
2. Call `redis.get(key)`.
3. **Cache hit:** Parse the JSON and return it immediately.
4. **Cache miss:** Proxy the request to the Creation Service, store the response with `redis.setex(key, TTL, JSON.stringify(response))`, and return it.

The TTL (e.g., 5 minutes) balances freshness with performance. Flight data changes infrequently enough that a 5-minute stale window is acceptable.

Cache invalidation: When admin updates a flight via the Creation Service, it publishes an event or the Searching Service relies on TTL expiry. For production, an explicit invalidation event on update would be added.

Q14. What is the difference between ioredis and the built-in redis client?

Answer: Both are Node.js Redis clients. **ioRedis** is preferred for production because: - Automatic **reconnect** with exponential backoff on connection drops. - Built-in support for **Redis Cluster** and **Redis Sentinel**. - First-class support for **pipelining** (batching multiple commands) and **Lua scripting**. - Promise-based API that plays well with `async/await`. - Handles **LOADING** errors during Redis startup gracefully.

The official `redis` npm package (v4) has similar features but `ioRedis` has a longer production track record and wider adoption at the time this project was built.

Q15. How would you handle cache stampede (thundering herd)?

Answer: A **cache stampede** happens when many requests simultaneously get a cache miss on the same key and all hit the database concurrently. Solutions:

1. **Mutex / locking:** Before querying the DB, acquire a distributed Redis lock (`SET lock:key 1 NX EX 5`). Only the process that holds the lock queries the DB; others wait or return stale data.
2. **Probabilistic early expiry (PER):** Refresh the cache slightly before it expires using a probability function, so refreshes are spread over time.
3. **Background refresh:** Keep serving stale data while an async job refreshes it. This is similar to stale-while-revalidate.

In the current implementation, the TTL is short enough that stampede risk is low, but for production I would add a Redis mutex.

5. Resilience — Circuit Breakers & Retries

Q16. What is the Circuit Breaker pattern and why did you use Opossum?

Answer: A **circuit breaker** wraps calls to external services. It has three states:

- **CLOSED:** Requests pass through normally.
- **OPEN:** After `n` consecutive failures the circuit opens and all requests fail-fast without calling the service. This protects the calling service from waiting on a broken dependency.
- **HALF-OPEN:** After a timeout, one test request is allowed through. If it succeeds, the circuit closes; if it fails, it reopens.

Opossum is a battle-tested Node.js circuit breaker library with configurable thresholds, timeouts, and fallback functions. I wrapped the Booking Service → Creation Service HTTP call in an Opossum breaker with:

```
const breaker = new CircuitBreaker(callCreationService, {
  timeout: 3000,           // fail if takes > 3s
  errorThresholdPercentage: 50,
  resetTimeout: 10000      // try again after 10s
});
breaker.fallback(() => ({ error: 'Flight service unavailable' }));
```

Q17. What is exponential backoff and where did you apply it?

Answer: Exponential backoff is a retry strategy where the wait time between retries **doubles** with each attempt (plus optional jitter):

```
retry 1: wait 100ms
retry 2: wait 200ms
retry 3: wait 400ms
retry 4: wait 800ms
```

This avoids overwhelming a recovering service with immediate retries (the thundering herd problem on retries).

In the project I applied exponential backoff on: 1. **RabbitMQ reconnect** — if the connection drops, the consumer reconnects with increasing delays. 2. **Failed email retries** — the cron job that retries failed notifications checks the `retryCount` field and applies backoff so recently-failed emails are not immediately retried.

6. Booking Service — Transactions, Idempotency & Cron

Q18. How do you prevent double bookings (race conditions on seats)?

Answer: Seat decrement is done inside a **MySQL transaction with pessimistic locking**:

```
BEGIN;
SELECT availableSeats FROM Flights WHERE id = ? FOR UPDATE;
-- if availableSeats >= requested:
UPDATE Flights SET availableSeats = availableSeats - ? WHERE id = ?;
```

```
INSERT INTO Bookings (...);  
COMMIT;
```

FOR UPDATE places an exclusive row lock so concurrent requests cannot read stale seat counts. Only one transaction can modify the row at a time; others wait. This prevents two users from booking the last seat simultaneously.

In the Sequelize ORM this is done using **transaction** and **lock: transaction.LOCK.UPDATE**.

Q19. What is idempotency and how is it implemented for payments?

Answer: Idempotency means that calling an operation multiple times produces the same result as calling it once. For payments this is critical — network timeouts can cause a client to retry a payment that was already processed.

Implementation: 1. Client generates a UUID **idempotencyKey** and sends it in the payment request body. 2. Before processing, the Booking Service checks the DB: **SELECT * FROM Bookings WHERE idempotencyKey = ?**. 3. If the record exists with **status = 'BOOKED'**, return the existing result without charging again. 4. If not, process the payment and store the **idempotencyKey** with the booking record.

The **idempotencyKey** column has a **UNIQUE constraint** in MySQL to prevent race conditions (two concurrent requests with the same key would get a DB conflict on the second insert).

Q20. How does the automatic booking expiry cron job work?

Answer: A **node-cron** job runs every minute:

```
cron.schedule('* * * * *', async () => {  
  const expiredBookings = await Booking.findAll({  
    where: {  
      status: 'INITIATED',  
      createdAt: { [Op.lt]: new Date(Date.now() - 5 * 60 * 1000) }  
    }  
  });  
  for (const booking of expiredBookings) {  
    await cancelBooking(booking); // restores seats, sets status = 'CANCELLED'  
  }  
});
```

The 5-minute window gives users enough time to complete payment. Cancelled bookings restore the reserved seats back to the flight record so other users can book them.

Q21. How do you cancel a specific booking and what side effects does cancellation have?

Answer: `POST /api/v1/bookings/cancel-booking` accepts a `bookingId`. The service:

1. Fetches the booking and validates the user owns it.
2. Checks that the booking is cancellable (not already `CANCELLED` or `EXPIRED`).
3. Calls the Creation Service (via circuit breaker) to increment seat count back: `PATCH /api/v1/flights/:id/seats`.
4. Updates the booking status to `CANCELLED`.
5. Optionally publishes a `booking_cancelled` event to RabbitMQ for a notification email.

All DB operations (status update + optional ticket creation) are wrapped in a Sequelize transaction to ensure atomicity.

7. Database Design — MySQL & Sequelize ORM

Q22. How did you design the database schema? What are the key relationships?

Answer: Each service has its own isolated database (Database-per-Service pattern):

API Gateway DB: - `Users` (id, email, passwordHash, role) - `Roles` (id, name) - `UserRoles` (userId, roleId) — many-to-many

Flights DB (Creation Service): - `Cities` → `Airports` (one-to-many) - `Airplanes` (id, modelNumber, capacity) - `Airlines` (id, name, description) - `Flights` (id, flightNumber, airplaneId, departureAirportId, arrivalAirportId, departureTime, arrivalTime, price, availableSeats, totalSeats)

Bookings DB: - `Bookings` (id, flightId, userId, seats, totalCost, status, idempotencyKey, createdAt) - `SeatBookings` (id, bookingId, seatNumber)

Notifications DB: - `Tickets` (id, subject, content, recipientEmail, status, retryCount, createdAt)

Q23. What database indexes did you add and why?

Answer: Strategic indexes on high-traffic query patterns:

Table	Column(s)	Reason
Bookings	<code>status, createdAt</code>	Cron job queries <code>status = 'INITIATED' AND createdAt < ?</code>
Bookings	<code>userId</code>	<code>GET /bookings/user/:userId</code>
Bookings	<code>idempotencyKey</code>	Idempotency check on every payment
Flights	<code>departureAirportId, arrivalAirportId, departureTime</code>	Primary search filter
Users	<code>email</code>	Login lookup by email
Tickets	<code>status</code>	Retry cron queries <code>status = 'FAILED'</code>

Without these indexes, a table scan would occur on every query. For the `Bookings` table with millions of rows, a composite index on `(status, createdAt)` reduces the cron job query from $O(n)$ to $O(\log n)$.

Q24. Why Sequelize and not a raw query builder or TypeORM?

Answer: Sequelize was chosen because: - Mature, well-documented, large community. - First-class migration and seeder CLI via `sequelize-cli`. - Built-in transaction support. - Model-level hooks (`beforeCreate`, `afterUpdate`) for business logic.

Trade-offs vs. alternatives: - **Prisma** would give stronger TypeScript types and a better DX, but the project is primarily JavaScript. - **Raw Knex.js** gives more SQL control but no model abstraction. - **TypeORM** is popular with TypeScript but has historically had more active issues.

For a pure JS microservices project, Sequelize's migration ecosystem (`db:migrate` , `db:seed:all`) is particularly useful for reproducible DB setup in CI.

8. Input Validation & Security

Q25. Why Zod instead of express-validator or Joi?

Answer: Zod offers: 1. **TypeScript-first schema inference** — the inferred type from a Zod schema can be used directly as the type of the validated data, eliminating duplication. 2. **Composability** — schemas can be combined, extended, and refined easily. 3. **Parse, don't validate** — `z.parse()` throws on invalid input and returns the correctly typed value, avoiding the two-step "validate then cast" pattern. 4. **No runtime dependencies** — Zod is zero-dependency.

The main trade-off is that Zod is newer than Joi and some developers are less familiar with it, but its superior TypeScript integration makes it worth the learning curve.

Q26. What security headers does Helmet set and why are they important?

Answer: Helmet sets HTTP response headers to protect against common web vulnerabilities:

Header	What it does
<code>Content-Security-Policy</code>	Restricts which scripts/styles/images can load
<code>X-Frame-Options</code>	Prevents clickjacking by blocking iframe embedding
<code>X-Content-Type-Options: nosniff</code>	Prevents MIME-type sniffing
<code>Strict-Transport-Security (HSTS)</code>	Forces HTTPS connections
<code>X-XSS-Protection</code>	Legacy XSS filter for older browsers
<code>Referrer-Policy</code>	Controls the Referer header

In the project, CSP is customized to allow `'unsafe-inline'` for styles (which would be tightened in production with a nonce).

Q27. How does bcrypt hashing work and what is the significance of salt rounds?

Answer: bcrypt is a password hashing function with a built-in salt. The algorithm: 1. Generates a random 16-byte salt. 2. Runs the Blowfish cipher `2^saltRounds` times (key stretching). 3. The output hash embeds the salt, so no separate storage is needed.

Salt rounds (also called the "cost factor") control computational cost. A salt round of 10 means $2^{10} = 1024$ iterations. Increasing by 1 doubles the time. At salt round 12, hashing takes ~300ms on modern hardware — slow enough to resist brute-force but fast enough for legitimate logins.

Why not MD5/SHA256? They are designed to be fast (for checksums), making brute-force trivial. bcrypt is intentionally slow for security.

9. Testing Strategy

Q28. Describe your testing pyramid for this project.

Answer:



Unit tests mock all external dependencies (DB, Redis, RabbitMQ) and test a single function in isolation:

- Service-layer functions (createBooking, makePayment, cancelBooking)
- Middleware functions (validateAuth, rateLimiter)
- Error utility classes

Integration tests use Supertest to make HTTP requests against the real Express app, but with mocked external calls: - Mock `sequelize.authenticate()` to avoid needing a real DB - Mock RabbitMQ channel with `jest.fn()` - Mock Redis client

Test count across services: 57 total (20 Gateway, 16 Booking, 10 Creation, 6 Searching, 5 Notifications).

Q29. How do you mock Redis and RabbitMQ in tests?

Answer: Redis (ioredis):

```
jest.mock('ioredis', () => {
  return jest.fn().mockImplementation(() => ({
    get: jest.fn().mockResolvedValue(null),
    setex: jest.fn().mockResolvedValue('OK'),
    incr: jest.fn().mockResolvedValue(1),
    expire: jest.fn().mockResolvedValue(1),
  }));
});
```

RabbitMQ (amqplib):

```
jest.mock('amqplib', () => ({
  connect: jest.fn().mockResolvedValue({
    createChannel: jest.fn().mockResolvedValue({
      assertQueue: jest.fn(),
      sendToQueue: jest.fn(),
      consume: jest.fn(),
    }),
  }),
}));
```

Using module-level mocks means the tests never need real infrastructure, making them fast and deterministic in CI.

10. CI/CD & Docker

Q30. Explain your GitHub Actions CI/CD pipeline.

Answer: The pipeline (`.github/workflows/ci.yml`) runs on every push to `main` / `develop` and on PRs to `main` . It has three jobs:

1. **Lint & Test:** Spins up MySQL 8.0, Redis 7, and RabbitMQ 3 as service containers. For each of the five services it installs dependencies, runs `npm test` , and collects coverage.
2. **Docker Build:** (Runs only on `main` after tests pass.) Builds Docker images for all five services using the per-service Dockerfiles.
3. **Security Audit:** Runs `npm audit` on each service to detect known CVEs in dependencies.

The service containers in GitHub Actions are a key feature — they let integration tests run against real infrastructure without needing a separate environment.

Q31. How is Docker Compose used and what problems does it solve?

Answer: `docker-compose.yml` at the root defines all eight containers: - `flights-mysql` — MySQL 8.0 with health check - `flights-redis` — Redis 7 Alpine - `flights-rabbitmq` — RabbitMQ 3 with management UI - Five application services (one per microservice)

Problems solved: 1. **Dependency ordering** — `depends_on` with `service_healthy` ensures the DB is up before the app starts. 2. **Networking** — All containers are on the same Docker network so they resolve each other by service name (`flights-mysql:3306`). 3. **Reproducibility** — `docker compose up --build` spins up the exact same environment on any machine. 4. **Volume persistence** — Named volumes persist MySQL data across restarts.

Q32. What is a multi-stage Docker build and why is it useful?

Answer: A multi-stage Dockerfile uses multiple `FROM` statements. Each stage produces a layer; only the final stage is included in the shipped image:

```
# Stage 1: build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

# Stage 2: runtime
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY src/ ./src/
CMD ["node", "src/index.js"]
```

This ensures `devDependencies`, build tools, and source files not needed at runtime are excluded from the final image, significantly reducing image size and attack surface.

11. Observability — Logging & Correlation IDs

Q33. How are correlation IDs implemented and why are they critical in microservices?

Answer: Correlation IDs enable **distributed tracing** — the ability to follow a single user request across multiple services in logs.

In the API Gateway, a middleware generates (or forwards an existing) `x-correlation-id` header:

```
app.use((req, res, next) => {
  req.correlationId = req.headers['x-correlation-id'] || uuid();
  res.setHeader('x-correlation-id', req.correlationId);
  next();
});
```

Every downstream HTTP call from the Gateway (and between services) propagates this header. The Winston logger in each service reads `req.correlationId` and includes it in every log line:

```
{ "level": "info", "message": "Booking created", "correlationId": "abc-123", "service": "booking-service" }
```

In production, a log aggregator (ELK Stack, Datadog) can filter by `correlationId` to see the complete trace of a request.

Q34. What is structured logging and why is it better than `console.log`?

Answer: **Structured logging** means emitting machine-parseable logs (typically JSON) instead of free-form strings.

Winston in this project emits:

```
{
  "timestamp": "2024-12-25T10:00:00Z",
  "level": "error",
  "message": "DB query failed",
  "service": "booking-service",
  "correlationId": "abc-123",
  "stack": "Error: connect ECONNREFUSED..."
}
```

Benefits over `console.log` : - **Searchable**: Can query `level=error AND service=booking-service` . - **Parseable by log aggregators** (ELK, Splunk, CloudWatch) without regex parsing. - **Consistent format** across all services. - **Log levels** (debug, info, warn, error) allow filtering in production. - **File transport** — logs are also written to files for persistence.

12. PixelPing — Chrome Extension (MV3)

Q35. What is PixelPing and how does email tracking work conceptually?

Answer: PixelPing is a Chrome browser extension that provides **email read receipts** — similar to WhatsApp blue ticks but for emails.

How email tracking works (pixel tracking): 1. When composing an email, the sender's email client (via the extension) inserts a 1×1 invisible image tag with a unique URL: `html ` 2. When the recipient opens the email, their email client loads the image, sending an HTTP GET request to the PixelPing backend. 3. The backend logs the request (timestamp, IP, user agent) against the unique ID and notifies the sender.

This approach requires no action from the recipient and works with any email client that renders HTML.

Q36. What is Manifest V3 and how does it differ from MV2?

Answer: Manifest V3 (MV3) is the current Chrome extension specification introduced in Chrome 88. Key changes from MV2:

Aspect	MV2	MV3
Background	Persistent background page	Service worker (ephemeral)
Request modification	<code>webRequest</code> (blocking)	<code>declarativeNetRequest</code> (declarative)
Scripting	<code>chrome.tabs.executeScript</code>	<code>chrome.scripting.executeScript</code>
Remote code	Allowed	Forbidden

Service worker vs. persistent background: MV3 background service workers are **ephemeral** — they start on demand and stop when idle. This means: - You cannot store state in global variables (it's lost when the worker sleeps). - All persistent state must go into `chrome.storage` or `IndexedDB`. - Ports/connections close and must be re-established.

In PixelPing, the background service worker listens for tab navigation events to detect when Gmail is opened and communicates with the content script via message passing.

Q37. How does message passing work between content scripts and the background?

Answer: Content scripts run in the **page context** (isolated from extension context). They can only communicate with the background via Chrome's message passing API.

One-way message (fire-and-forget):

```
// content script → background
chrome.runtime.sendMessage({ type: 'PING_FROM_CONTENT' });

// background listens
chrome.runtime.onMessage.addListener((message, sender, sendResponse) => {
  if (message.type === 'PING_FROM_CONTENT') {
    console.log('[PixelPing] Content script pinged from tab', sender.tab?.id);
  }
});
```

PixelPing message types: - `PING_FROM_CONTENT` — content script confirms it's alive -
`CLIENT_CONTENT_SCRIPT_PING` — background polls content script -
`PIXELPING_COMPOSE_TRACKING_TOGGLE` — user toggles tracking in Gmail compose

All message types are typed in `shared/types/` to provide compile-time safety.

Q38. How does the Gmail content script inject UI into compose windows?

Answer: Gmail dynamically renders the DOM — compose windows are injected asynchronously. The content script uses a **MutationObserver** to watch for DOM changes:

```
const observer = new MutationObserver((mutations) => {
  for (const mutation of mutations) {
    for (const node of mutation.addedNodes) {
      if (node instanceof HTMLElement && node.matches('.compose-button-area')) {
        injectTrackingToggle(node);
      }
    }
  }
});
```

```
    }  
  });  
  observer.observe(document.body, { childList: true, subtree: true });
```

When a compose area appears, `injectTrackingToggle()` appends an "Enable Tracking" button. This approach is fragile to Gmail DOM changes (Google may change class names), which is a known limitation of Gmail integrations.

Q39. Why use Vite to build a Chrome extension? What is the build output?

Answer: Vite provides: - Fast HMR (Hot Module Replacement) during development. - TypeScript compilation without a separate `tsc` step. - Tree-shaking for smaller bundles. - Plugin system — `vite-plugin-web-extension` can handle MV3 specifics.

The Vite config in PixelPing produces:

```
dist/  
  background/background.js    - Service worker bundle  
  content/gmail-integration.js - Content script bundle  
  manifest.json               - Copied as-is  
  assets/icons/               - Icons
```

These are the exact files Chrome expects when loading an unpacked extension. The build separates background and content bundles because Chrome loads them in different contexts and they have different permissions.

Q40. What are the limitations you faced with PixelPing and how would you address them?

Answer: (Note: The user acknowledged "said more than done" for PixelPing — answer honestly and show awareness of what remains.)

What was implemented: - MV3 Chrome extension scaffold with TypeScript - Background service worker detecting Gmail tab navigation - Content script injecting a UI toggle into Gmail compose - Typed message passing between background and content - Node.js + TypeScript backend with security middleware (Helmet, CORS, rate limiting, input sanitization) - Google OAuth setup (googleapis library) for user authentication flow

Known limitations / next steps: 1. **No pixel tracking implementation** — The tracking URL generation and open detection backend route are not implemented yet. Would use a UUID stored in MongoDB with `mongoose` and a lightweight `GET /track/open?id=:uuid`

endpoint. 2. **Compose toggle has no persistence** — The toggle state is lost when Gmail re-renders. Fix: `chrome.storage.sync.set({ trackingEnabled: true })`. 3. **No popup UI** — A React popup for the extension toolbar button is scaffolded but not implemented. 4. **No API call from extension** — The content script does not yet call the backend to generate tracking pixels.

13. PixelPing — Backend (TypeScript + Express + MongoDB)

Q41. Why TypeScript for the backend and what are its practical benefits?

Answer: TypeScript provides: 1. **Static typing** catches type errors at compile time, not runtime. 2. **IDE autocomplete and navigation** — editors can show available properties on objects. 3. **Refactoring safety** — renaming a field updates all usages. 4. **Interfaces and generics** — define request/response shapes explicitly. 5. **Discriminated unions** — model different message types safely (critical for message passing in the extension).

In the PixelPing backend, `tsconfig-paths` allows `@routes`, `@config`, `@middlewares` path aliases, making imports cleaner. The trade-off is an added build step (`tsc`) and a steeper initial setup.

Q42. Why MongoDB (Mongoose) for PixelPing instead of MySQL like the flights app?

Answer: PixelPing's data model is **document-oriented**: - Tracking events are arbitrary JSON objects (varying user agents, metadata). - Email compose sessions can have nested arrays of recipient objects. - Schema can evolve without migrations as the product is early-stage.

MongoDB's flexible schema is a better fit than MySQL's rigid table schema for this type of unstructured/semi-structured data. Mongoose adds schema validation and model-level middleware on top of MongoDB.

In the flights app, the data is highly relational (flights reference airports, which reference cities; bookings reference flights and users), making MySQL's joins and referential integrity a natural fit.

Q43. How does the Google OAuth flow work in PixelPing?

Answer: (The `googleapis` package is listed as a dependency — OAuth2 integration is planned/partially implemented.)

OAuth2 Authorization Code Flow: 1. **Backend generates an auth URL** using `OAuth2Client.generateAuthUrl()` with scopes (e.g., `gmail.send` or `gmail.compose`). 2. **User is redirected** to Google's consent page. 3. **Google redirects back** to the configured callback URL with a `code`. 4. **Backend exchanges the code** for access + refresh tokens: `OAuth2Client.getToken(code)`. 5. **Tokens are stored** securely (encrypted in MongoDB, not plain text). 6. For subsequent API calls, the access token is used; if expired, the refresh token silently fetches a new one.

The purpose of Google OAuth in PixelPing is to authenticate users and potentially use the Gmail API to inject tracking pixels into outgoing emails server-side (as an alternative to the extension).

Q44. Explain the `sanitizeInput` middleware in the PixelPing backend.

Answer: `sanitizeInput` is a middleware that runs on all routes and strips or encodes potentially malicious characters from user inputs to prevent **XSS (Cross-Site Scripting)** and **NoSQL injection**:

```
export const sanitizeInput = (req: Request, res: Response, next: NextFunction) => {
  if (req.body) {
    req.body = sanitizeObject(req.body);
  }
  if (req.query) {
    req.query = sanitizeObject(req.query) as ParsedQs;
  }
  next();
};

function sanitizeObject(obj: any): any {
  if (typeof obj === 'string') return obj.replace(/[<>]/g, '').trim();
  if (typeof obj === 'object') {
    // remove keys that start with $ (NoSQL injection)
    for (const key in obj) {
      if (key.startsWith('$')) delete obj[key];
      else obj[key] = sanitizeObject(obj[key]);
    }
  }
  return obj;
}
```

NoSQL injection is a MongoDB-specific concern — an attacker can send `{ "email": { "$gt": "" } }` to bypass authentication if not sanitized.

14. General Backend & Node.js Concepts

Q45. What is the Node.js event loop and why is it important to understand?

Answer: Node.js is **single-threaded** and uses an event loop to handle concurrency. The event loop has phases:

1. **Timers** — executes `setTimeout` / `setInterval` callbacks.
2. **I/O callbacks** — handles completed I/O operations (DB queries, file reads).
3. **Idle/Prepare** — internal use.
4. **Poll** — retrieves new I/O events; block here if queue is empty.
5. **Check** — executes `setImmediate` callbacks.
6. **Close callbacks** — e.g., `socket.on('close', ...)`.

Why it matters for my projects: All DB queries, Redis calls, and HTTP requests are **async** (non-blocking). The event loop processes other requests while waiting for I/O responses. If I accidentally make a CPU-intensive synchronous operation (e.g., a large bcrypt hash synchronously), it **blocks the event loop**, stalling all other requests. That's why `bcrypt.hash()` (async) is used, not `bcrypt.hashSync()`.

Q46. What is the difference between `Promise.all` and `Promise.allSettled`?

Answer: | `Promise.all` | `Promise.allSettled` | |---|---|---| | Resolves when | **All** promises resolve | **All** promises settle (resolve OR reject) | | Rejects when | **Any** promise rejects | Never (always resolves) | | Use case | When all must succeed | When you need results of all, even failures |

Example in the project: When fetching user bookings and their associated flight details in parallel:

```
// Use Promise.all if all must succeed
const [booking, flight] = await Promise.all([fetchBooking(id), fetchFlight(flightId)]);

// Use Promise.allSettled if you want partial results
const results = await Promise.allSettled(bookings.map(b => fetchFlight(b.flightId)));
const flights = results.filter(r => r.status === 'fulfilled').map(r => r.value);
```

Q47. How does Express middleware work and what is the order of execution?

Answer: Express middleware is a function with signature `(req, res, next)`. Middleware functions are executed in the **order they are registered** with `app.use()`.

```
Request → Helmet → CORS → Body Parser → Auth → Rate Limiter → Route Handler → Error Handler
```

Key rules: - Call `next()` to pass to the next middleware; don't call it to short-circuit. - Call `next(err)` to skip to the **error-handling middleware** (4 arguments: `(err, req, res, next)`). - Error handlers must be registered **last**.

In the project, the auth middleware must run **before** the rate limiter (which keys by `userId`) and before the RBAC middleware (which needs `req.user`).

Q48. What is the difference between `async/await` and `callbacks`? Why prefer `async/await`?

Answer: Callbacks use the Node.js convention `(error, result) => {}`. They lead to **callback hell** (deeply nested code) and error handling is manual.

Promises flatten the nesting with `.then().catch()` chaining.

Async/await is syntactic sugar over Promises, making async code look synchronous:

```
// Callback (old)
db.query('SELECT ...', (err, results) => {
  if (err) return next(err);
  processResults(results, (err2, final) => { ... });
});

// Async/await (clean)
try {
  const results = await db.query('SELECT ...');
  const final = await processResults(results);
} catch (err) {
  next(err);
}
```

Async/await is preferred because it is more readable, easier to debug (stack traces are meaningful), and error handling with `try/catch` is familiar.

15. HR / Behavioural Round

Q49. Tell me about a technical challenge you faced in the Flights Booking App and how you solved it.

Answer: "One of the trickiest challenges was handling the seat booking race condition. When multiple users try to book the last seat simultaneously, a naive implementation would let both succeed (overselling).

I solved this using a MySQL transaction with `SELECT ... FOR UPDATE`, which places a pessimistic row lock on the flight record. Only one transaction can hold the lock at a time — the other waits. This guaranteed that seat count updates are serialized and prevent double-booking.

I also wrote an integration test that simulates two concurrent booking requests for the last seat to verify only one succeeds. This gave me confidence the solution worked before deploying it."

Q50. How would you improve the Flights Booking App if given more time?

Answer: Priority improvements: 1. **Event sourcing / saga pattern** for distributed transactions — replace the current compensating approach with a formal saga orchestrator. 2. **Service mesh (Istio/Linkerd)** for mutual TLS between services, automatic retries, and service-level metrics without code changes. 3. **Kubernetes deployment** — replace Docker Compose with K8s manifests + Helm charts for production-grade orchestration. 4. **Rate limiting with token bucket** — replace the current fixed-window Redis counter with a token bucket algorithm for smoother throttling. 5. **OpenAPI/Swagger documentation** — generate interactive API docs from Zod schemas. 6. **Distributed tracing with Jaeger/Zipkin** — replace manual correlation ID propagation with OpenTelemetry auto-instrumentation. 7. **React frontend** — currently API-only; a booking UI would demonstrate the full stack.

Q51. Why did you choose this tech stack?

Answer: "I chose Node.js because its async, event-driven model is ideal for I/O-heavy microservices — most operations are waiting on DB, Redis, or HTTP, not CPU.

MySQL for the relational data in the Flights and Bookings services because flight schedules, seat counts, and booking records have strict relational integrity requirements (foreign keys, transactions).

RabbitMQ over Kafka because for this scale (thousands of messages, not millions), RabbitMQ is simpler to operate and its consumer ACK model maps naturally to the email notification pattern.

Redis because it is both a cache (Searching Service) and a distributed data structure store (rate limiter counters) — two use cases with one infrastructure component.

For PixelPing, I switched to MongoDB because email tracking events are semi-structured and the schema will evolve rapidly in an early-stage product."*

Q52. How do you approach debugging a production issue in a microservices system?

Answer: My approach (OODA loop): 1. **Observe:** Check health endpoints (`/health`) on all services. Look at dashboards for error rate spikes. Check RabbitMQ management UI for queue depths. 2. **Orient:** Reproduce the issue with the correlation ID from the error report. Filter logs across all services by that `x-correlation-id` . 3. **Decide:** Identify which service's log shows the first error. Is it a timeout, a 4xx client error, or a 5xx? 4. **Act:** - If it's a circuit breaker open state, check the downstream service health. - If it's a DB error, check connection pool exhaustion or deadlocks. - If it's a RabbitMQ queue depth growing, the consumer may be crashing — check the DLQ.

Correlation IDs are the most critical tool here — without them, tracing a request across five services is like finding a needle in a haystack.

End of Interview Q&A

Note: Questions marked as "conceptual" (PixelPing, some design questions) are based on industry knowledge and project intent. For implementation-specific questions about PixelPing, be transparent about what is scaffolded vs. fully implemented.